



```
In [16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import xgboost as xgb
import shap
import warnings
warnings.filterwarnings('ignore')
```

```
In [17]: df = pd.read_csv('Walmart_cleaned.csv')
df_clean = df.dropna()
print("Размер датасета:", df.shape)
print("\nПервые 5 строк:")
print(df.head())
print("\nТипы данных:")
print(df.dtypes)
```

Размер датасета: (6435, 42)

Первые 5 строк:

	Store	Date	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	\
0	1	2010-02-05	1643690.90	0	42.31	2.572	
1	1	2010-02-12	1641957.44	1	38.51	2.548	
2	1	2010-02-19	1611968.17	0	39.93	2.514	
3	1	2010-02-26	1409727.59	0	46.63	2.561	
4	1	2010-03-05	1554806.68	0	46.50	2.625	

	CPI	Unemployment	Year	Month	...	Sales_Lag_4_scaled	\
0	211.096358	8.106	2010	2	...	0.913717	
1	211.242170	8.106	2010	2	...	0.913717	
2	211.289143	8.106	2010	2	...	0.913717	
3	211.319643	8.106	2010	2	...	0.913717	
4	211.350143	8.106	2010	3	...	1.072626	

	Sales_Lag_52_scaled	Sales_Mean_4_scaled	Sales_Mean_12_scaled	\
0	0.893433	1.089638	1.096539	
1	0.893433	1.088058	1.094946	
2	0.893433	1.069310	1.076043	
3	0.893433	0.967771	0.973668	
4	0.893433	0.927265	0.965571	

	Sales_Trend_scaled	Season_Autumn	Season_Spring	Season_Summer	\
0	0.001890	False	False	False	
1	-0.018089	False	False	False	
2	-0.314697	False	False	False	
3	-1.619010	False	False	False	
4	-0.448216	False	True	False	

	Season_Winter	Store_encoded
0	True	0
1	True	0
2	True	0
3	True	0
4	False	0

[5 rows x 42 columns]

типы данных:

Store	int64
Date	object
Weekly_Sales	float64
Holiday_Flag	int64
Temperature	float64
Fuel_Price	float64
CPI	float64
Unemployment	float64
Year	int64
Month	int64
Week	int64
DayOfYear	int64
IsWeekend	int64

```

Season          object
Sales_Lag_1     float64
Sales_Lag_4     float64
Sales_Lag_52    float64
Sales_Mean_4    float64
Sales_Mean_12   float64
Weekly_Sales_mean float64
Weekly_Sales_min float64
Weekly_Sales_max float64
Temperature_mean float64
Fuel_Price_mean float64
CPI_mean        float64
Unemployment_mean float64
Sales_Trend     float64
Temperature_scaled float64
Fuel_Price_scaled float64
CPI_scaled      float64
Unemployment_scaled float64
Sales_Lag_1_scaled float64
Sales_Lag_4_scaled float64
Sales_Lag_52_scaled float64
Sales_Mean_4_scaled float64
Sales_Mean_12_scaled float64
Sales_Trend_scaled float64
Season_Autumn   bool
Season_Spring   bool
Season_Summer    bool
Season_Winter    bool
Store_encoded   int64
dtype: object

```

```

In [19]: target = 'Weekly_Sales'
         features = ['Temperature', 'Fuel_Price', 'CPI', 'Unemployment',
                    'Holiday_Flag', 'Year', 'Month', 'Week',
                    'Sales_Lag_52_scaled', 'Weekly_Sales_mean'
                    ]

         X = df[features]
         y = df[target]
         X = pd.get_dummies(X, drop_first=True)
         X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

In [ ]: from math import sqrt
         rf = RandomForestRegressor(
            n_estimators=200,
            random_state=42,
            max_depth=10,
            n_jobs=-1
         )
         rf.fit(X_train, y_train)

         y_pred_rf = rf.predict(X_test)

```

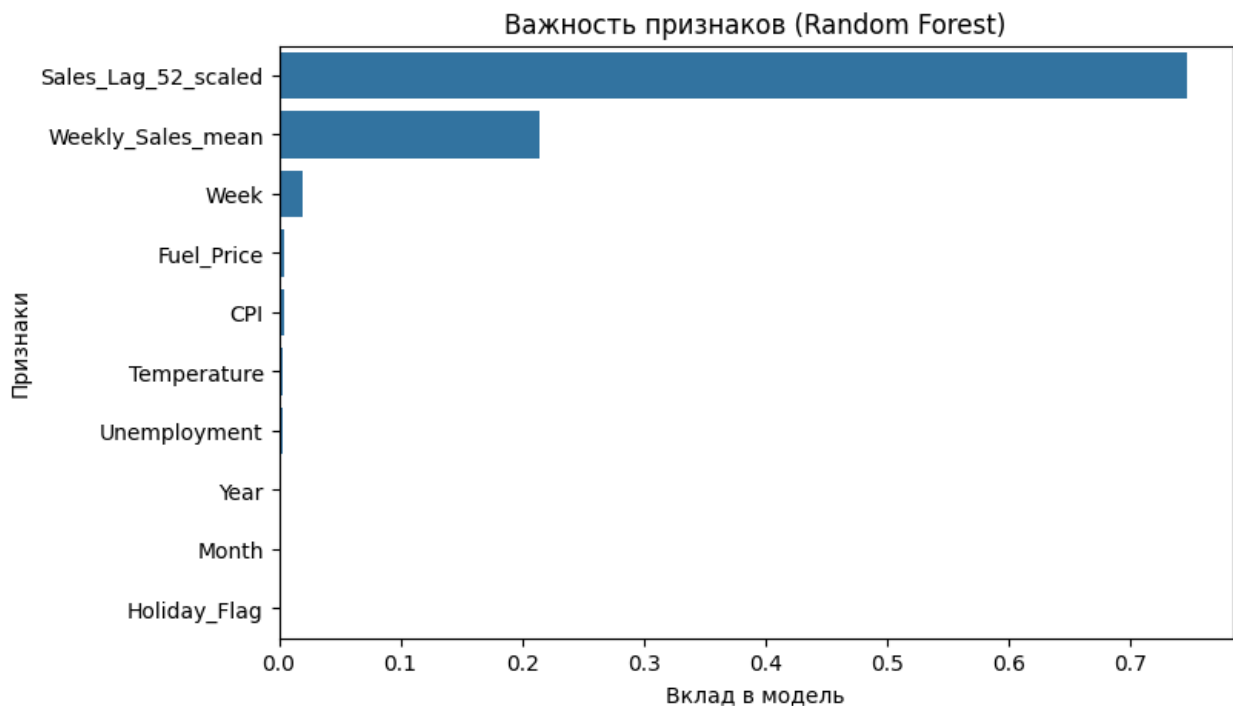
```
print("Random Forest:")
print("R²:", r2_score(y_test, y_pred_rf))
print("RMSE:", sqrt(mean_squared_error(y_test, y_pred_rf)))
```

Random Forest:
R²: 0.9925378357230296
RMSE: 0.08692752930962894

```
In [21]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

feature_importances = pd.DataFrame({
    'feature': X_train.columns,
    'importance': rf.feature_importances_
}).sort_values(by='importance', ascending=False)

plt.figure(figsize=(8,5))
sns.barplot(x='importance', y='feature', data=feature_importances)
plt.title("Важность признаков (Random Forest)")
plt.xlabel("Вклад в модель")
plt.ylabel("Признаки")
plt.show()
```



```
In [ ]: from xgboost import XGBRegressor

xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=6,
    random_state=42
)
```

```

X_train = X_train.select_dtypes(include=['int', 'float']).copy()
X_test = X_test.select_dtypes(include=['int', 'float']).copy()

xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)

print("XGBoost:")
print("R²:", r2_score(y_test, y_pred_xgb))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_xgb)))

```

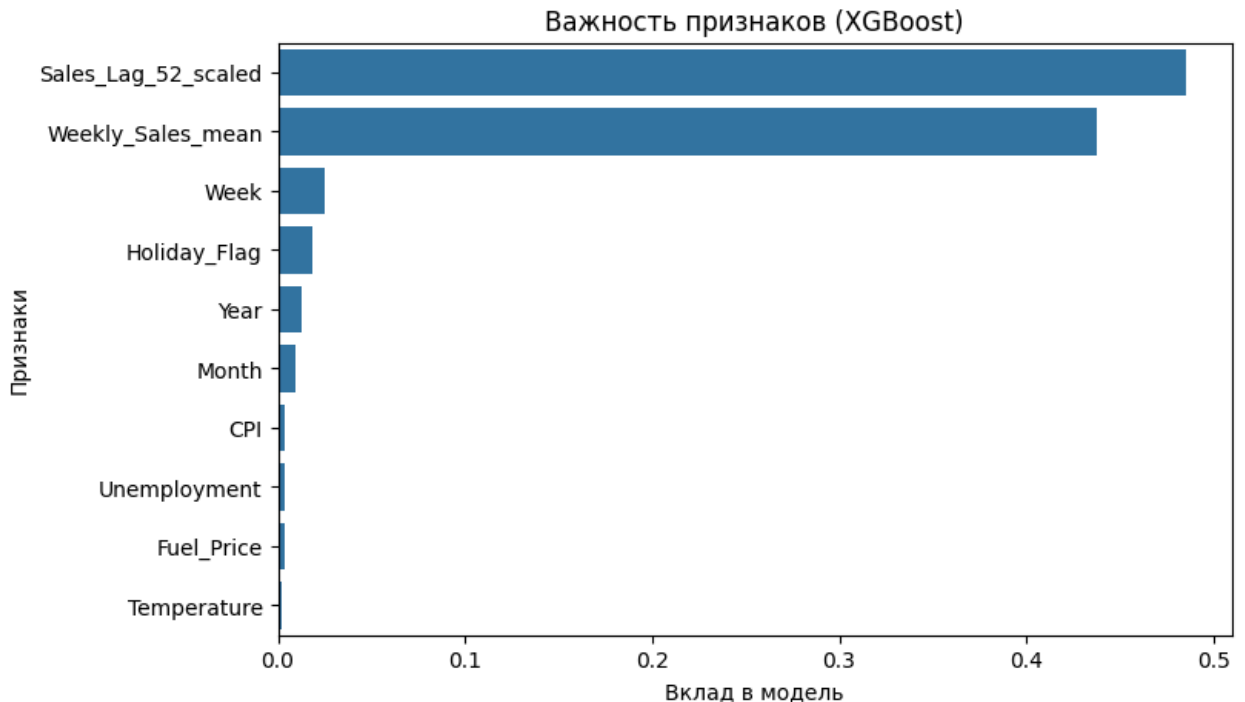
XGBoost:
 R²: 0.996693234460605
 RMSE: 0.05732788284016933

```

In [23]: xgb_importance = pd.DataFrame({
          'feature': X_train.columns,
          'importance': xgb.feature_importances_
        }).sort_values(by='importance', ascending=False)

plt.figure(figsize=(8,5))
sns.barplot(x='importance', y='feature', data=xgb_importance)
plt.title("Важность признаков (XGBoost)")
plt.xlabel("Вклад в модель")
plt.ylabel("Признаки")
plt.show()

```



```

In [24]: import numpy, shap, xgboost
print("numpy:", numpy.__version__)
print("shap:", shap.__version__)
print("xgboost:", xgboost.__version__)

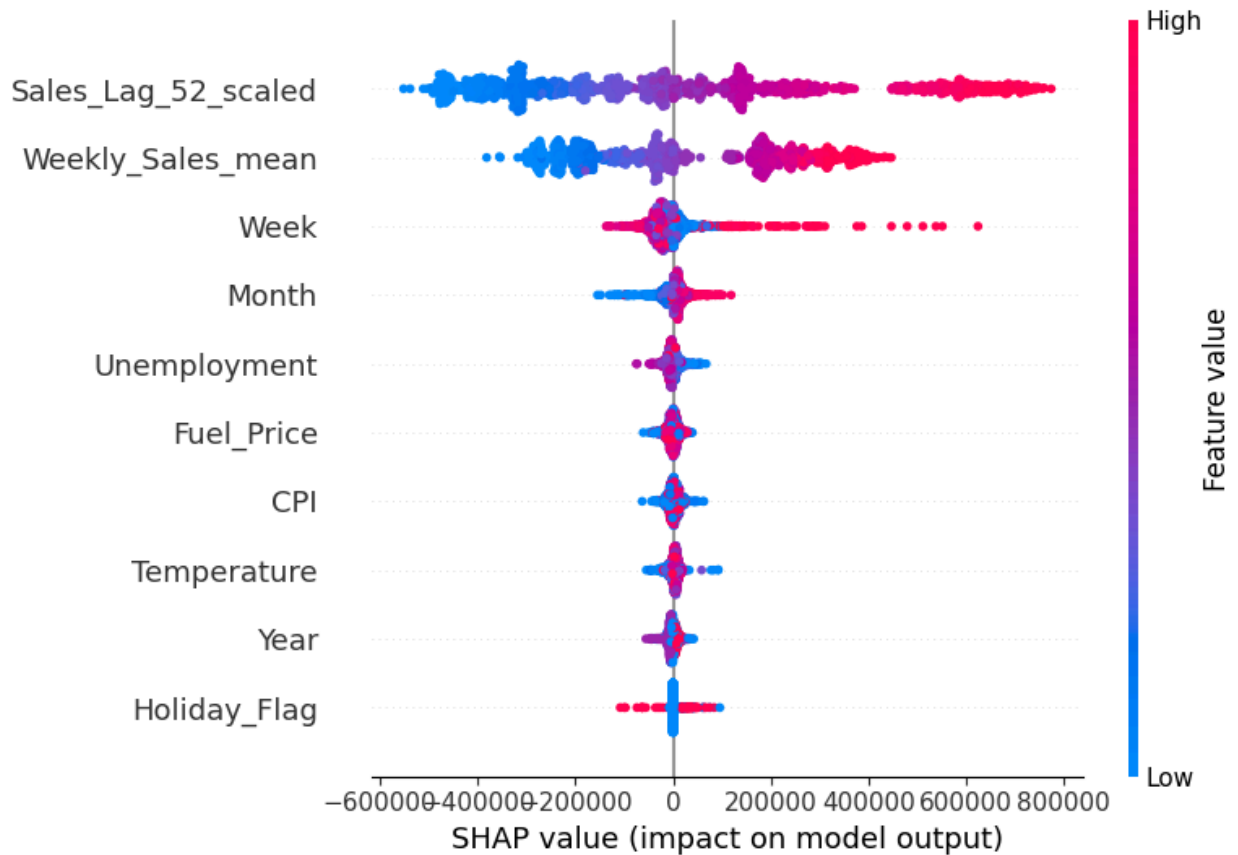
```

numpy: 2.2.3
shap: 0.50.0
xgboost: 3.1.2

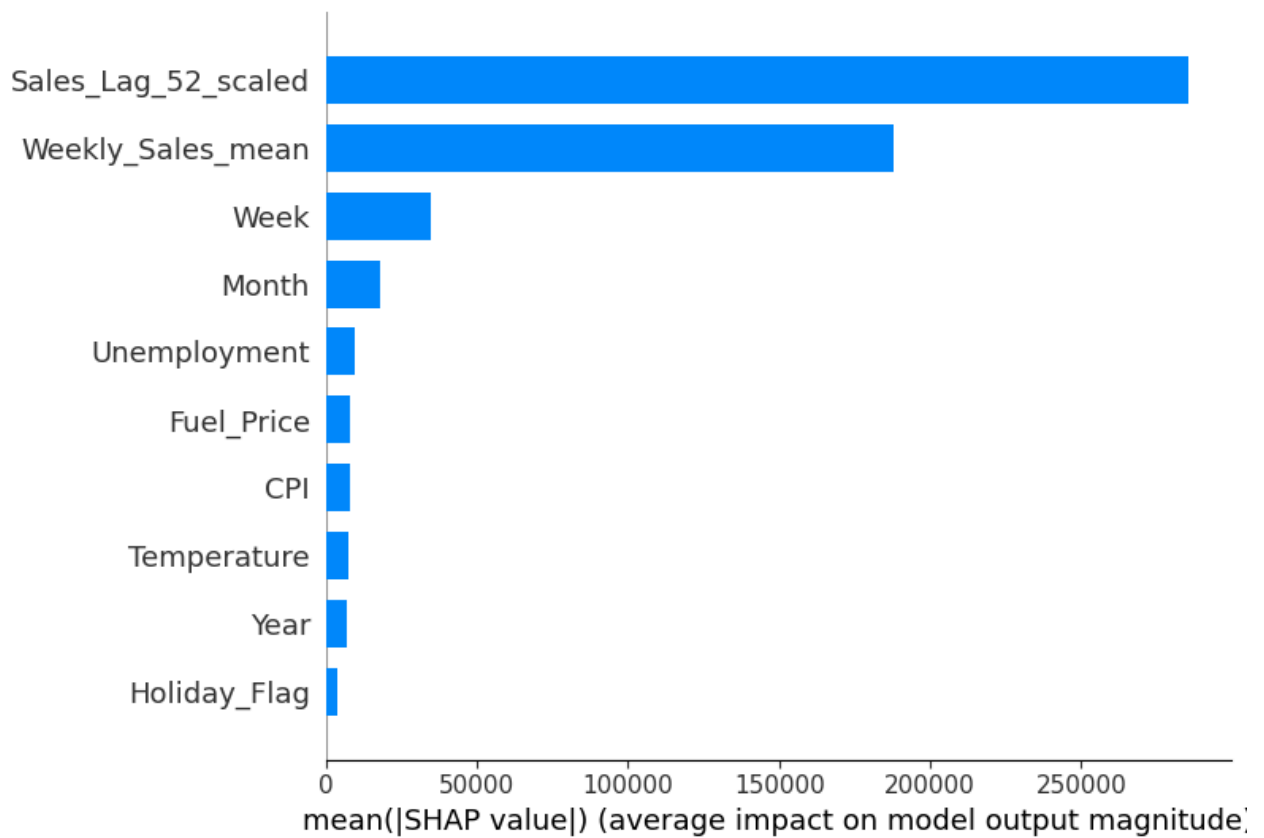
In [25]: `import shap`

```
booster = xgb.get_booster()
explainer = shap.TreeExplainer(booster, model_output='raw')
shap_values = explainer.shap_values(X_test)

shap.summary_plot(shap_values, X_test)
```



In [26]: `shap.summary_plot(shap_values, X_test, plot_type="bar")`



```
In [27]: shap.dependence_plot("Sales_Lag_52_scaled", shap_values, X_test)
```

